

```

"""
DecodeLabs Project 3: SQL Data Analysis
Author: [Your Name]
Description: Loads the cleaned e-commerce
dataset into a SQLite database
              and runs 10 business-focused
SQL queries covering SELECT, WHERE,
              ORDER BY, GROUP BY, HAVING,
COUNT, SUM, and AVG.
              Results are printed and
exported to a CSV report.
"""

```

```

import sqlite3
import pandas as pd

#
=====
=====
# STEP 1: LOAD CSV INTO SQLITE DATABASE
#
=====
=====
df = pd.read_csv("cleaned_dataset.csv")

conn = sqlite3.connect("orders.db")
df.to_sql("orders", conn,
if_exists="replace", index=False)
print("Database created: orders.db")
print(f"Table 'orders' loaded with

```

```

{len(df)} rows\n")

#
=====
=====
# Helper function to run and display each
query
#
=====
=====
results_log = []

def run_query(query_id, title, sql, conn):
    print(f"{' '*60}")
    print(f"QUERY {query_id}: {title}")
    print(f"{' '*60}")
    print(f"SQL:\n{sql.strip()}\n")
    result = pd.read_sql_query(sql, conn)
    print(result.to_string(index=False))
    print()
    results_log.append({"query_id":
query_id, "title": title, "sql":
sql.strip()})
    return result

#
=====
=====
# QUERY 1: Basic SELECT – View all columns
for first 10 orders
#
=====
=====
run_query(1, "Preview First 10 Orders", """)

```

```
SELECT OrderID, Date, Product, Quantity,  
UnitPrice, OrderStatus, TotalPrice  
FROM orders  
LIMIT 10  
""", conn)
```

```
#  
=====
```

QUERY 2: WHERE – Filter only Delivered orders

```
#  
=====
```

run_query(2, "All Delivered Orders", ""
SELECT OrderID, Date, Product, TotalPrice
FROM orders
WHERE OrderStatus = 'Delivered'
ORDER BY TotalPrice DESC
LIMIT 10
""", conn)

```
#  
=====
```

QUERY 3: WHERE + comparison – High-value orders above \$2000

```
#  
=====
```

run_query(3, "High-Value Orders (TotalPrice
> \$2000)", ""
SELECT OrderID, Product, Quantity,
UnitPrice, TotalPrice, OrderStatus

```

FROM orders
WHERE TotalPrice > 2000
ORDER BY TotalPrice DESC
""", conn)

#
=====
=====
# QUERY 4: COUNT + GROUP BY – Total orders
per product
#
=====
=====
run_query(4, "Total Orders by Product
(COUNT)", "")
SELECT Product,
        COUNT(*) AS Total_Orders
FROM orders
GROUP BY Product
ORDER BY Total_Orders DESC
""", conn)

#
=====
=====
# QUERY 5: SUM + GROUP BY – Total revenue
per product
#
=====
=====
run_query(5, "Total Revenue by Product
(SUM)", "")
SELECT Product,
        SUM(TotalPrice) AS Total_Revenue,

```

```

COUNT(*) AS Order_Count
FROM orders
GROUP BY Product
ORDER BY Total_Revenue DESC
""", conn)

#
=====
=====
# QUERY 6: AVG + GROUP BY – Average order
value per payment method
#
=====
=====
run_query(6, "Average Order Value by
Payment Method (AVG)", "")
SELECT PaymentMethod,
        ROUND(AVG(TotalPrice), 2) AS
Avg_Order_Value,
        COUNT(*) AS Total_Orders
FROM orders
GROUP BY PaymentMethod
ORDER BY Avg_Order_Value DESC
""", conn)

#
=====
=====
# QUERY 7: GROUP BY + ORDER BY – Orders by
referral source
#
=====
=====
run_query(7, "Orders and Revenue by

```

```

Referral Source", ""
SELECT ReferralSource,
        COUNT(*) AS Total_Orders,
        ROUND(SUM(TotalPrice), 2) AS
Total_Revenue,
        ROUND(AVG(TotalPrice), 2) AS
Avg_Order_Value
FROM orders
GROUP BY ReferralSource
ORDER BY Total_Orders DESC
"", conn)

```

```

#

```

```

=====
=====

```

```

# QUERY 8: GROUP BY + HAVING – Products
with more than 170 orders

```

```

#

```

```

=====
=====

```

```

run_query(8, "High-Volume Products – More
Than 170 Orders (HAVING)", "")
SELECT Product,
        COUNT(*) AS Total_Orders
FROM orders
GROUP BY Product
HAVING COUNT(*) > 170
ORDER BY Total_Orders DESC
"", conn)

```

```

#

```

```

=====
=====

```

```

# QUERY 9: WHERE + GROUP BY – Cancelled

```

```

orders by product
#
=====
=====
run_query(9, "Cancelled Orders Count by
Product", "")
SELECT Product,
        COUNT(*) AS Cancelled_Orders
FROM orders
WHERE OrderStatus = 'Cancelled'
GROUP BY Product
ORDER BY Cancelled_Orders DESC
"", conn)

#
=====
=====
# QUERY 10: Advanced – Revenue contribution
% per product
#
=====
=====
run_query(10, "Revenue Contribution % by
Product", "")
SELECT Product,
        ROUND(SUM(TotalPrice), 2) AS
Product_Revenue,
        ROUND(SUM(TotalPrice) * 100.0 /
(SELECT SUM(TotalPrice) FROM orders), 2) AS
Revenue_Pct
FROM orders
GROUP BY Product
ORDER BY Revenue_Pct DESC
"", conn)

```

```
#
=====

=====
# STEP 2: EXPORT QUERY LOG TO CSV
#
=====

=====

log_df = pd.DataFrame(results_log)
log_df.to_csv("sql_query_log.csv",
index=False)

conn.close()
print("="*60)
print("All queries complete.")
print("Database: orders.db")
print("Query log: sql_query_log.csv")
```